



Walrus Track Problem Statement

AI agents today are powerful, but still fundamentally stateless and fragmented. They complete tasks in isolation, lose context across sessions, and struggle to share knowledge across tools, teams, or workflows. Memory is often tied to a single app, model, or device — making agent systems brittle, hard to scale, and difficult to trust.

As agents evolve from simple assistants to autonomous, long-running systems, they need a more durable foundation:

- the ability to store and retrieve memory across sessions
- share context across agents and workflows
- and access data that is portable, persistent, and not locked into a single platform

This track challenges you to rethink how agentic systems are built by using Walrus as a Verifiable Data Platform for AI.

What you'll build

Build functional AI agents or agentic workflows (single or multi-agent) in any domain — from finance to productivity to gaming — that demonstrate:

- Long-term memory using [persistent, verifiable memory for agents](#)
- Persistent data and file access using Walrus (directly or via a file management interface)

- Integrations and tooling that make it easier for developers to adopt Walrus or MemWal (Walrus Memory) in agentic systems

To guide you, we're especially interested in:

- Long-running workflows where agents track state over time (e.g., research agents, trading agents, monitoring systems)
- Multi-agent coordination, such as negotiation, task delegation, or step-by-step execution across agents
- Artifact-driven workflows, where agents generate, store, and reuse files like datasets, logs, reports, or intermediate outputs

For integrations and tooling, think along the lines of:

- adding persistent memory to existing agent frameworks or tools (e.g., plugins or adapters to use Walrus directly, or to use MemWal as the Walrus Memory layer)
- creating workflow orchestration layers that combine memory, messaging, and execution across agents with Walrus as the underlying storage foundation
- enabling cross-tool or cross-agent memory sharing, where different systems can read/write to the same context stored on Walrus
- building interfaces or developer tools that make it easier to inspect, debug, or manage agent memory and data stored on Walrus

Your project could be:

- a user-facing agent or multi-agent system
- a developer tool or framework integration
- or a new interface for interacting with persistent AI memory and data

What we're looking for

We're not just looking for demos — we're looking for working systems that show:

- how agents become more useful when they can remember and build over time
- how workflows improve when data is shared, durable, and portable
- and how developers can move beyond fragile, siloed memory setups

The goal is to push toward a future where AI agents are not just reactive tools, but persistent, collaborative systems powered by a reliable data layer.

References to use:

- [Walrus docs](#)
 - [Getting started](#)
 - [CLI / HTTP API / Typescript SDK](#)
 - [Public aggregators and publishers](#)
- [Walrus Sites docs](#)
 - [Install the site-builder CLI](#)
 - [Publish a site](#)
- [MemWal \(Walrus Memory\) docs](#)
 - [MemWal \(Walrus Memory\) Playground](#) - create an account and a delegate key for your agent
 - [MemWal \(Walrus Memory\) Github Repo](#) - includes sample apps, skills etc.
- [Seal docs](#) - privacy layer for Walrus and MemWal
- [Sui Stack Messaging](#) - messaging tooling that uses Walrus for storage & recovery and Seal for privacy

Join the Walrus Builder Group

For questions, discussions, and direct support from the Walrus team, join the official [*Walrus Telegram group*](#)

For idea validation and project discussion, join our [*Walrus Discord #developers channel*](#)

